

8.3

Инкремент и декремент

00:00–00:17

Введение

Привет!

В этой теме вы познакомитесь с так называемыми операциями инкремента и декремента, с их разновидностями, а также со сложными случаями, которые могут встретиться не только в коде, но и в качестве задач на собеседовании.

00:17–03:29

Инкремент и декремент

Инкремент — это **увеличение числа на единицу**. Вы уже знаете, что существуют операции сложения и вычитания с присвоением, и их можно записывать проще: `+=` или `-=`. Так вот, инкремент **`value++`** — это более простая запись увеличения на единицу с присвоением.

Все три записи эквивалентны, и все они увеличивают число `value` на единицу.

Инкремент

```
int value = 54;  
value = value + 1; //55  
value += 1; //56  
value++; //57
```

Существует также и **декремент** — **уменьшение числа на единицу**. Он записывается, как и инкремент, но вместо `++` пишется `--`.

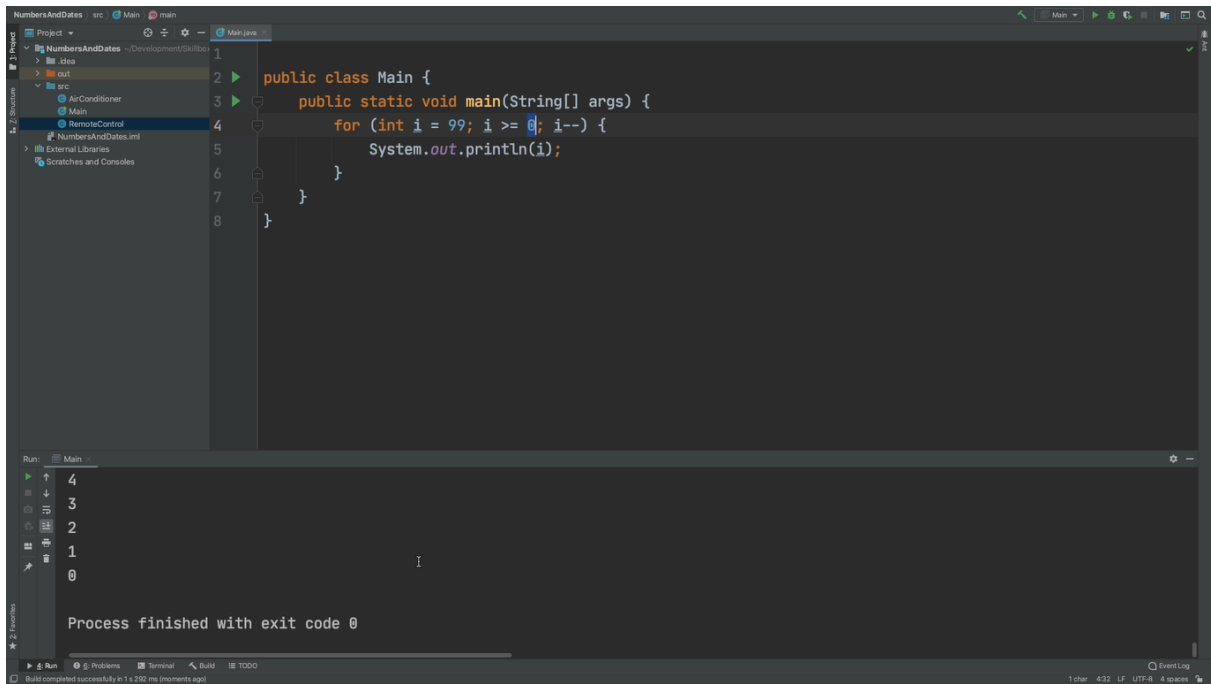
Декремент

```
int value = 54;  
value = value - 1; //53  
value -= 1; //52  
value--; //51
```

Инкремент и декремент могут быть полезны в реальной жизни. Например, нужно перебрать числа от 0 до 100. Вы можете использовать инкремент в цикле, собственно, именно так и принято писать в циклах:



Если нужно перебрать числа в обратном порядке, можете использовать декремент, и заданное число будет уменьшаться на единицу:



```
1 public class Main {
2     public static void main(String[] args) {
3         for (int i = 99; i >= 0; i--) {
4             System.out.println(i);
5         }
6     }
7 }
8 }
```

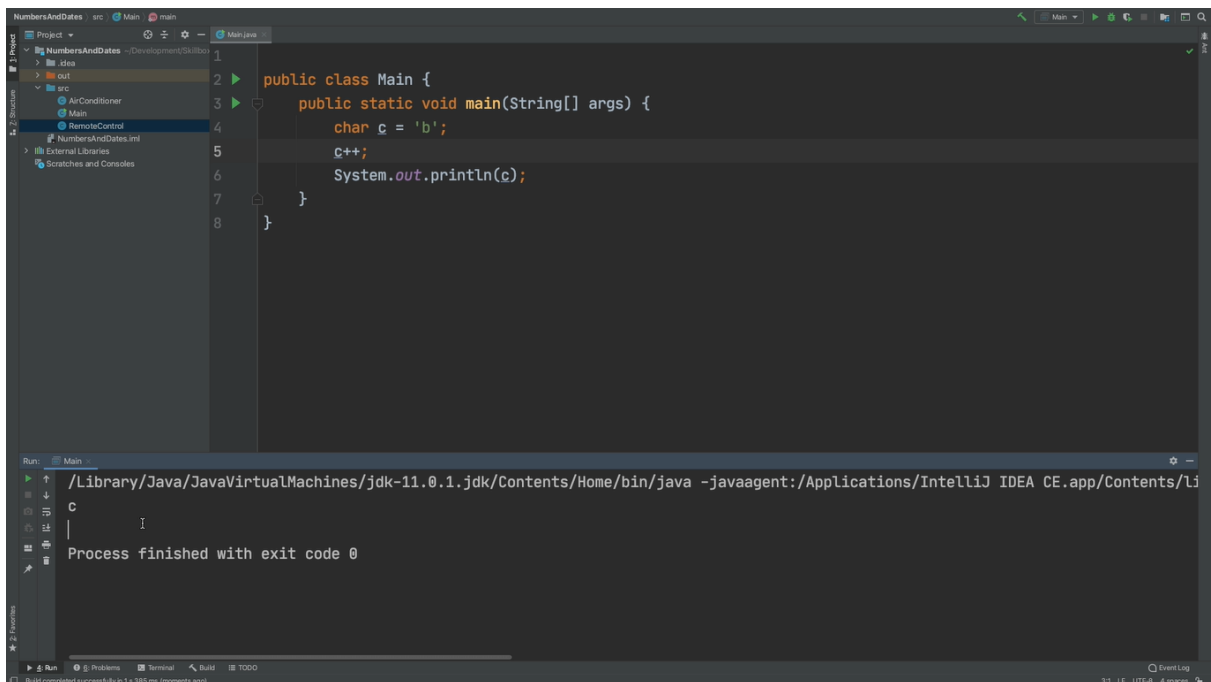
Run: Main

4
3
2
1
0

Process finished with exit code 0

Обратите внимание, что в данном примере мы перебираем числа до нуля, включая ноль.

Инкрементировать и декрементировать можно не только числа, но и символы. Например, если создадим символ `b` и увеличим его на единицу (а значит и числовое значение, которое соответствует этому символу), то получим следующий по алфавиту символ `c`.



```
1 public class Main {
2     public static void main(String[] args) {
3         char c = 'b';
4         c++;
5         System.out.println(c);
6     }
7 }
8 }
```

Run: Main

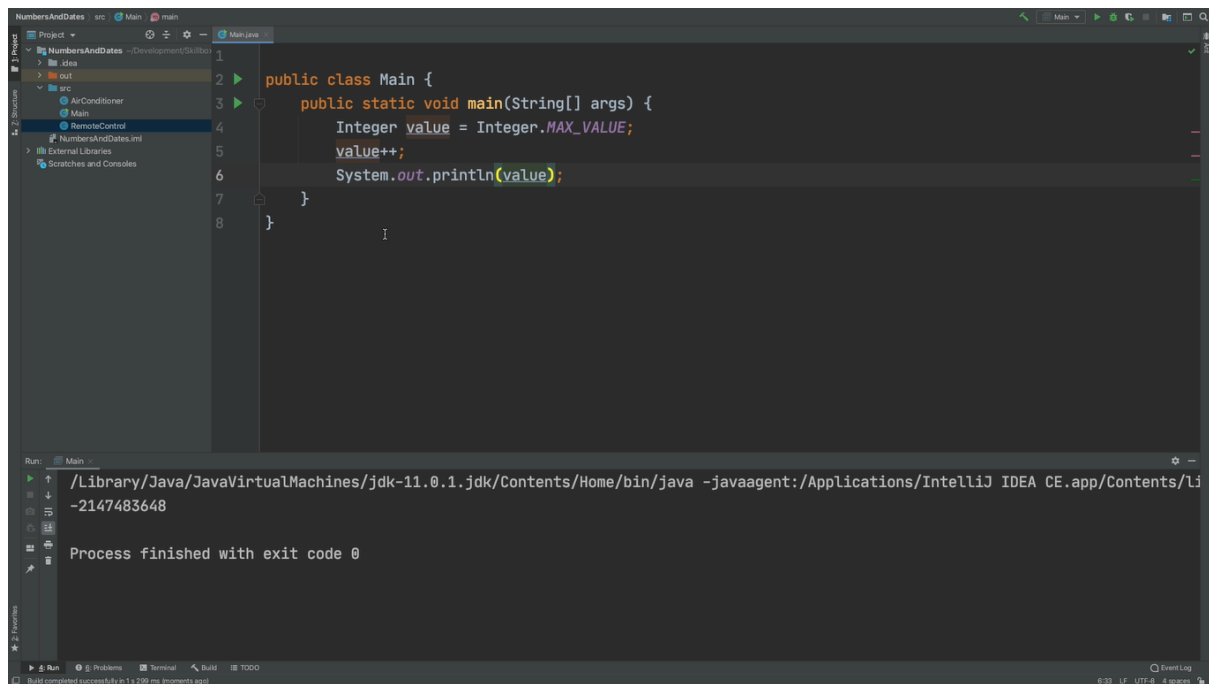
/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/Li

c

Process finished with exit code 0

Это происходит потому, что в кодировке эти символы располагаются друг за другом и числовой код символа с на единицу больше числового кода символа b.

Ещё один важный момент — если вы увеличите число, которое и так имеет максимально допустимое значение, например, `Integer.MAX_VALUE`, то произойдёт так называемое переполнение и число примет минимальное значение:



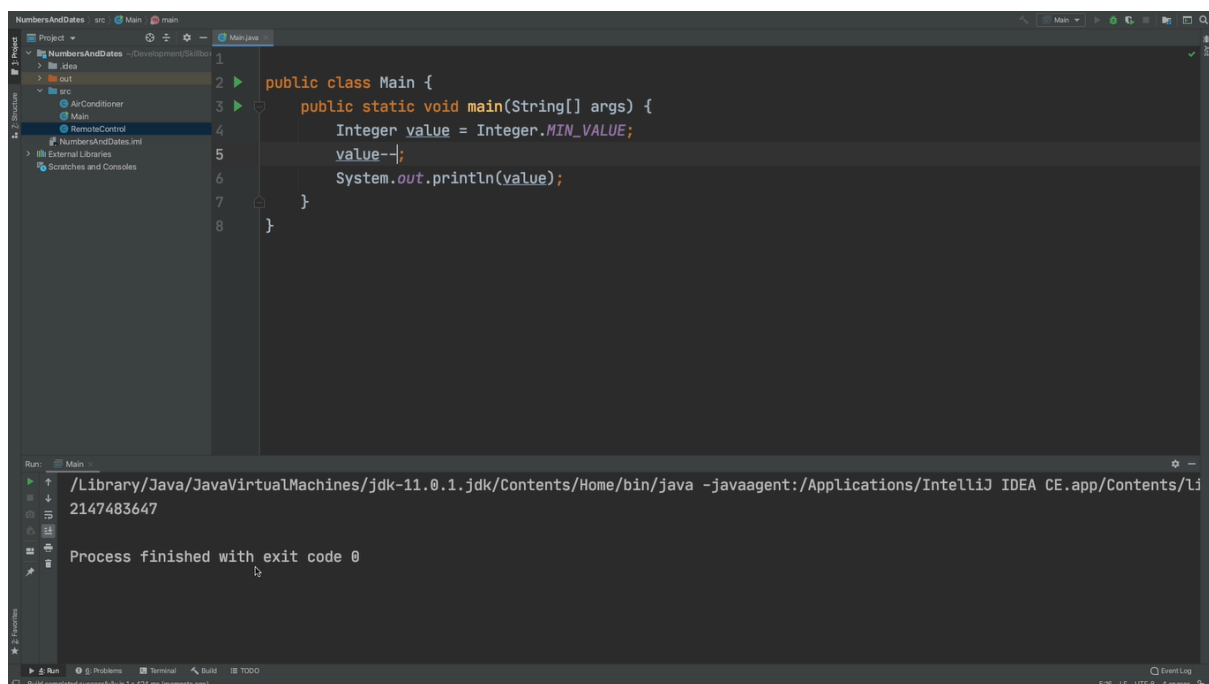
```
1 public class Main {
2     public static void main(String[] args) {
3         Integer value = Integer.MAX_VALUE;
4         value++;
5         System.out.println(value);
6     }
7 }
8
```

Run: Main

/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/Li
-2147483648

Process finished with exit code 0

То же самое верно и для декремента.



```
1 public class Main {
2     public static void main(String[] args) {
3         Integer value = Integer.MIN_VALUE;
4         value--;
5         System.out.println(value);
6     }
7 }
8
```

Run: Main

/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/Li
2147483647

Process finished with exit code 0

03:29–10:00

Пост- и преинкремент

Есть ещё одна особенность инкремента и декремента, которая может быть полезна при оптимизации кода. О ней часто спрашивают на собеседованиях — это так называемые **постинкремент** и **преинкремент**, а также **постдекремент** и **предекремент**. Начнём с инкремента.

Если напишем `value++`, то число `value` будет увеличено на единицу, но при этом само выражение `value++` вернёт то значение, которое было до инкремента. В данном случае значение переменной `post` будет таким же, как и значение переменной `value`. Это так называемый **постинкремент**.

Постинкремент

```
int value = 38;  
int post = value++; //38
```

Если же напишем `++` перед словом `value`, то сначала произойдёт операция инкремента, и только затем этим выражением будет возвращено значение. В данном случае переменная `pre` будет равна 39. Это **преинкремент**.

Преинкремент

```
int value = 38;  
int pre = ++value; //39
```

Запомнить это очень просто:

- если “++” стоит после имени переменной, это так называемый постинкремент — сначала значение переменной возвращается, а потом увеличивается;

- если же “++” стоит перед переменной, то это преинкремент — сначала значение переменной увеличивается, а потом увеличенное значение возвращается.

Аналогично при операции декремента. В данном случае переменная `post` будет равна исходному значению переменной `value`, поскольку это **постдекремент**, а сама переменная `value` после выполнения кода уменьшится на единицу.

Постдекремент

```
int value = 38;  
int post = value--; //38
```

А в следующем случае — это **предекремент**, поэтому сначала значение переменной уменьшится, а потом уменьшенное значение вернётся.

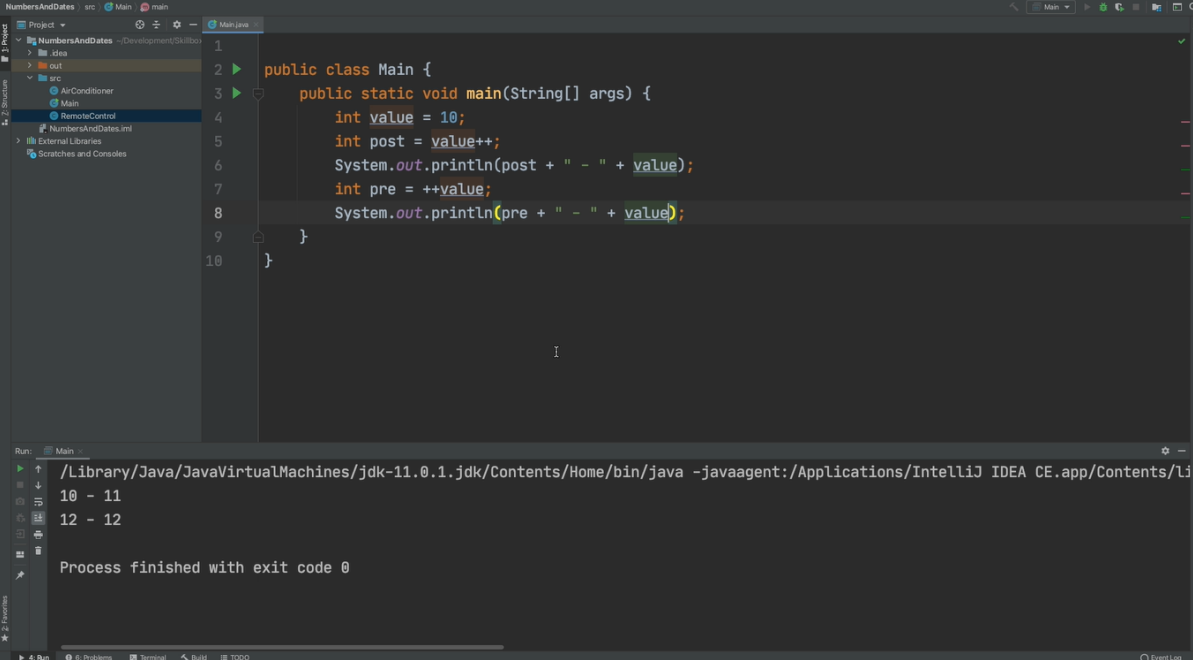
Предекремент

```
int value = 38;  
int pre = --value; //37
```

Давайте посмотрим, как преинкремент и постинкремент, а также предекремент и постдекремент применяются в коде, и чем они могут быть полезны.

Создадим число `value = 10`. Сначала посчитаем постинкремент, выведем значение переменной `post` и значение переменной `value`. Затем посчитаем преинкремент и также выведем значение

переменной pre и значение переменной value. Запускаем код:



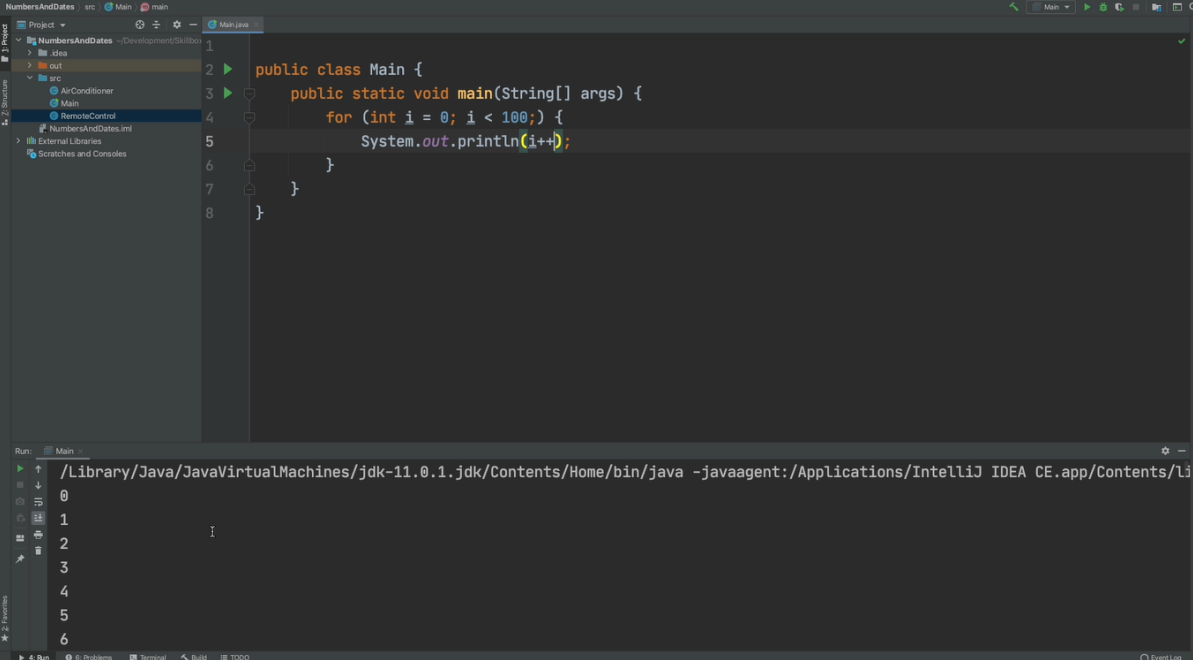
```
1 public class Main {
2
3     public static void main(String[] args) {
4         int value = 10;
5         int post = value++;
6         System.out.println(post + " - " + value);
7         int pre = ++value;
8         System.out.println(pre + " - " + value);
9     }
10 }
```

Run: Main

```
/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/Li
10 - 11
12 - 12

Process finished with exit code 0
```

Можно написать и другой пример: цикл for, в котором мы не напишем последнее условие (i++) и который запишем в System.out.println. Получим такой же результат, как если бы этот оператор i++ находился в цикле.



```
1 public class Main {
2
3     public static void main(String[] args) {
4         for (int i = 0; i < 100; ) {
5             System.out.println(i++);
6         }
7     }
8 }
```

Run: Main

```
/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/Li
0
1
2
3
4
5
6
```

Если поменяем постинкремент на преинкремент, то сначала значение увеличивается на единицу, а уже потом увеличенное значение возвращается.

Соответственно, получим результат со сдвигом на единицу — это будут числа от 1 до 100.



The screenshot shows an IDE with a project named 'NumbersAndDates'. The 'Main.java' file is open, showing the following code:

```
1 public class Main {
2     public static void main(String[] args) {
3         for (int i = 0; i < 100; i++) {
4             System.out.println(++i);
5         }
6     }
7 }
8 }
```

The 'Run' console at the bottom shows the output of the program:

```
96
97
98
99
100
Process finished with exit code 0
```

В такие моменты нужно быть очень внимательным при написании кода — сдвиг на единицу может привести к исключению, ошибке во всей программе и к тому, что программа вылетит.

Чем удобны пре-/постинкремент и пре-/постдекремент? Например, если вы пишете цикл и используете в нём `i++` или `++i`, то в данном случае разницы никакой не будет — оба этих цикла будут эквивалентны.

Примеры

```
for (int i = 0; i < 100; i++) {
    //some code here
}

for (int i = 0; i < 100; ++i) {
    //some code here
}
```

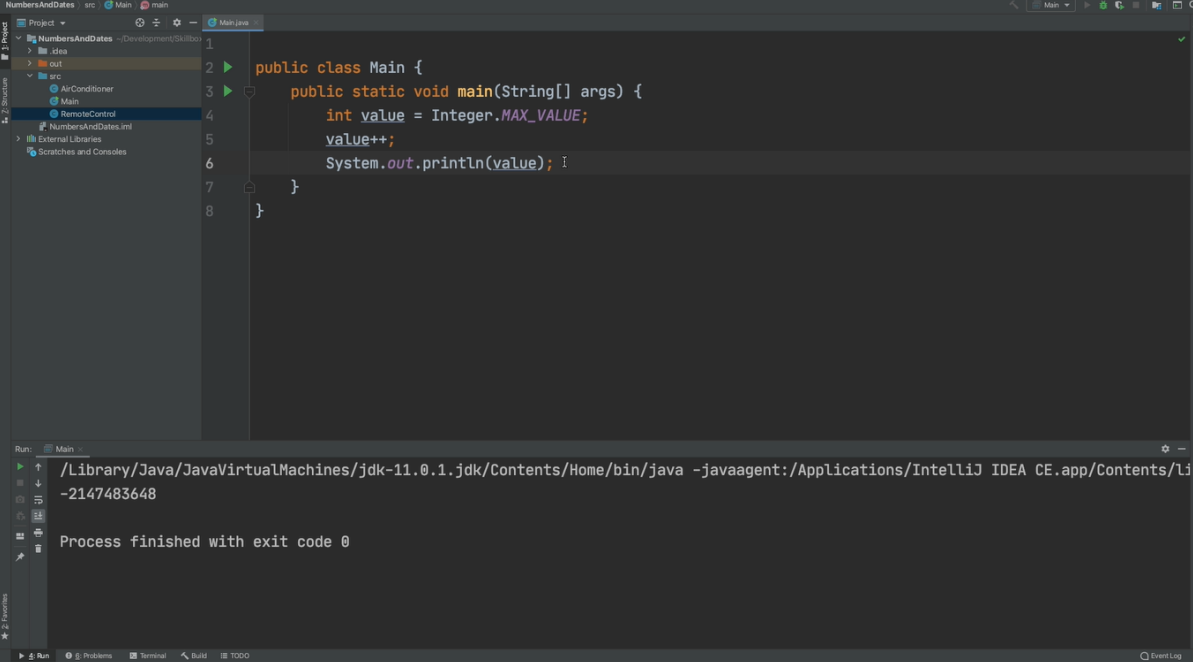
Оба цикла эквивалентны!

Так происходит из-за того, что это выражение в цикле используется без присвоения другой переменной — в цикле используется собственное значение переменной `i`, которая и так и так увеличивается на единицу.

Теперь посмотрим, чем может быть удобен преинкремент.

Допустим, есть некое значение `integer max value`. Увеличиваем это значение на единицу и дальше выводим результат в консоль.

Очевидно, что получится минимально возможное значение переменной класса `integer`:



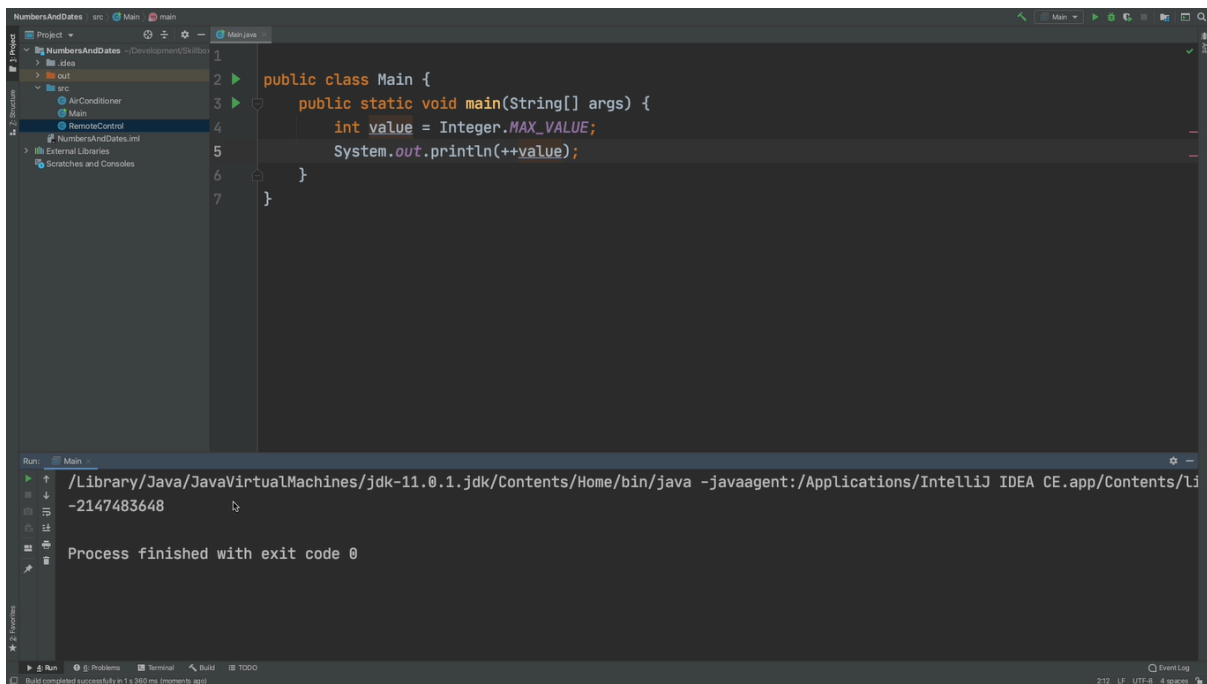
The screenshot shows the IntelliJ IDEA IDE with a project named "NumbersAndDates". The "Main.java" file is open, displaying the following code:

```
1 public class Main {
2     public static void main(String[] args) {
3         int value = Integer.MAX_VALUE;
4         value++;
5         System.out.println(value);
6     }
7 }
8
```

The "Run" tab at the bottom shows the execution output:

```
/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/Li
-2147483648
Process finished with exit code 0
```

Можно упростить этот код и внести `++value` в `System.out.println`, сократив код на одну строку:



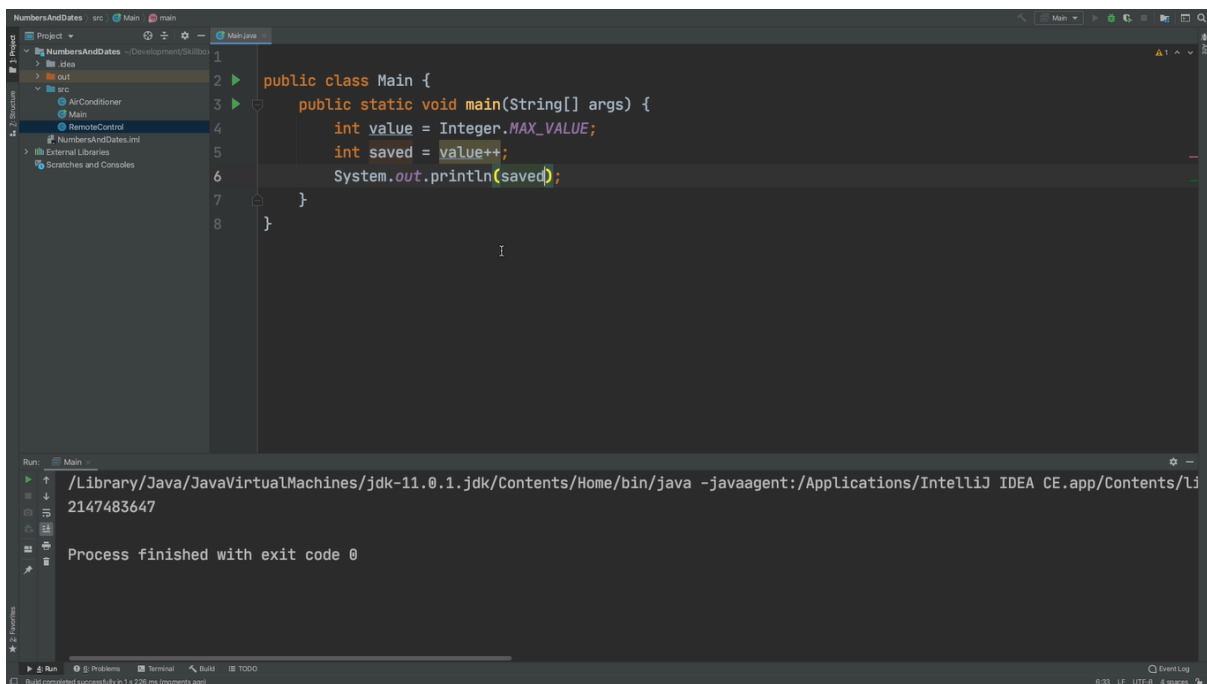
The screenshot shows the IntelliJ IDEA IDE with a project named "NumbersAndDates". The "Main.java" file is open, displaying the following code:

```
1 public class Main {
2     public static void main(String[] args) {
3         int value = Integer.MAX_VALUE;
4         System.out.println(++value);
5     }
6 }
7
```

The "Run" tab at the bottom shows the execution output:

```
/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/Li
-2147483648
Process finished with exit code 0
```

Если вам нужно сохранить значение до инкремента, то воспользуйтесь постинкрементом:



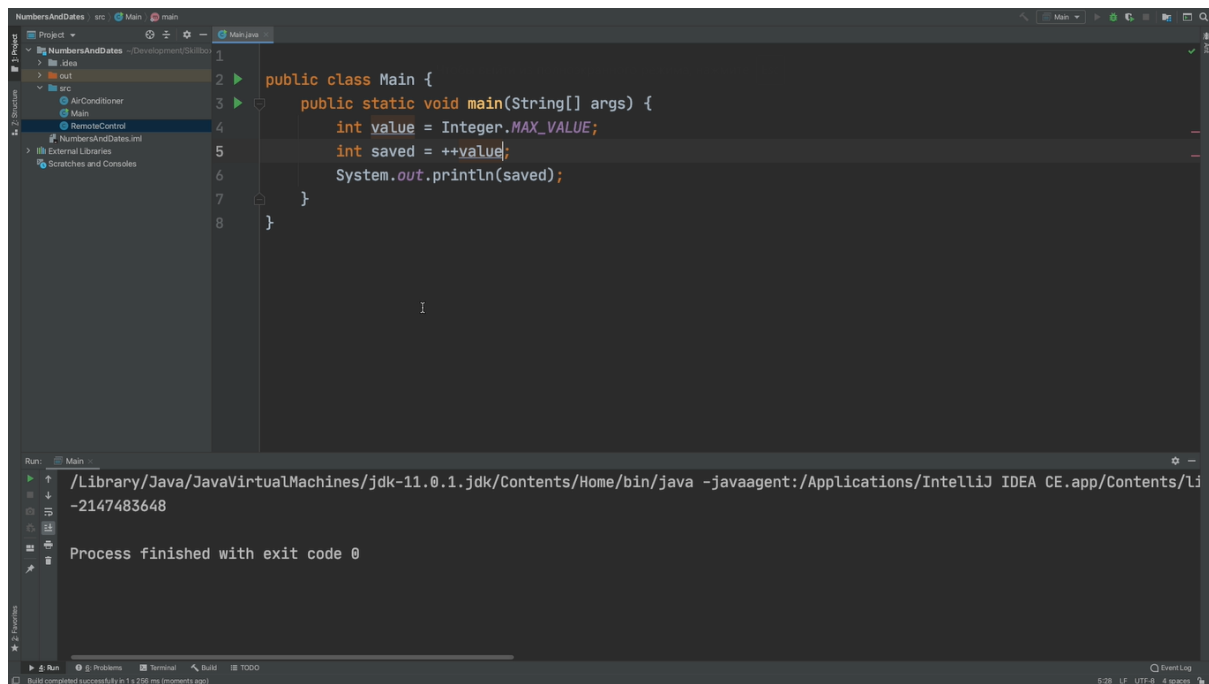
The screenshot shows the IntelliJ IDEA IDE with the same project "NumbersAndDates". The "Main.java" file is open, displaying the following code:

```
1 public class Main {
2     public static void main(String[] args) {
3         int value = Integer.MAX_VALUE;
4         int saved = value++;
5         System.out.println(saved);
6     }
7 }
8
```

The "Run" tab at the bottom shows the execution output:

```
/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/Li
2147483647
Process finished with exit code 0
```

Если же вы хотите получить значение после того, как произойдёт инкремент, то воспользуйтесь преинкрементом:



Пре-/постинкремент, как и пре-/постдекремент могут быть полезны при оптимизации кода, чтобы сделать его более компактным.

Повторим пройденный материал:

Пре- и постинкремент и пре- и постдекремент

	Нужно получить значение переменной после операции	Нужно получить значение переменной до операции
...увеличения	Преинкремент ++value	Постинкремент value++
...уменьшения	Предекремент --value	Постдекремент value--

10:00–11:07

Сложные случаи

Разберём несколько более сложных случаев применения пре-/постинкремента и пре-/постдекремента:

```
int x = 50;  
int value = x++ + ++x;
```

Как будет происходить вычисление:

1. На первом этапе вычисляется значение выражения “x++”, при этом x станет равен 51, а само значение выражения “x++” будет равно 50.
2. Затем выполняется второе выражение — “++x”. Сначала произойдёт инкремент, то есть значение x, которое было равно 51, увеличится на единицу — 52, и далее вернётся именно оно.
3. Потом результаты этих выражений суммируются, и итоговый результат будет равен 102.

Такие задачи часто задают на собеседовании, чтобы проверить ваше умение интерпретировать код в голове. Это очень важное умение, поскольку оно позволяет читать и понимать написанный программный код быстрее.

Сложные случаи

```
int x = 50;  
int value = x++ + ++x;
```

Шаг	Чему равен “x”	Чему равно значение
x++	51	50
++x	52	52
+	52	102

11:07–11:35

Итоги

Итак, в этой теме вы познакомились с операциями инкремента и декремента, а также с вариантами этих операций: пре-/постинкрементом и пре-/постдекрементом.

Следующая тема «Преобразование чисел между различными типами». Вы узнаете, что такое автоматическое неявное преобразование чисел, в каких случаях оно происходит, а в каких случаях возможно только явное преобразование.